



POLITÉCNICA

Universidad Politecnica de Madrid

Bases de Datos II

Practica MongoDB: Jotun's Lair

Yixuan Lu Guo
Shengkai Zhu

21 de mayo de 2026

Índice

1. Introducción	2
2. Tarea 1: Implementacion de los endpoints	2
2.1. GET /user/{email}	2
2.2. GET /room/{room_id}	3
2.3. GET /dungeon/{dungeon_id}	4
2.4. POST /comment	5
2.5. DELETE /monster/{monster_id}	6
3. Tarea 2: Consultas analíticas	6
3.1. Query 1: Monstruos en las 10 habitaciones con más comentarios Bug	6
3.2. Query 2: Mazmorras con comentarios Hint por encima de la media	7
3.3. Query 3: Estadísticas por mazmorra	8
3.4. Query 4: Mazmorras por tipo de monstruo	11
3.5. Query 5: Tipo de monstruo más común por mazmorra	11
3.6. Query 6: Comentarios de la habitación con encuentro de mayor nivel medio	12
3.7. Query 7: Habitaciones outliers según ratio oro/nivel	13
4. Tarea 3: Patrones de diseño	14
4.1. Patrón Subset (Subconjunto)	15
4.2. Patron Computed (Valores precalculados)	16
4.3. Patron Outlier (Datos atípicos)	16
4.4. Justificación global	16
5. Tarea 4: Cambio de esquema	16
5.1. A. Consulta de extraccion y exportacion a JSON	17
5.2. B. Crear la coleccion Hints y limpiar las originales	17
5.3. C. Adaptación de los endpoints	17
5.3.1. POST /comment (nuevo)	17
5.3.2. GET /room/{room_id} (nuevo)	18
5.3.3. GET /user/{email} (nuevo)	19
5.3.4. GET /dungeon/{dungeon_id} (nuevo)	19
5.3.5. Impacto general del cambio	21
6. Conclusiones	22

1. Introducción

Esta práctica aborda la migración de la base de datos relacional que da soporte a la wiki del videojuego **Jotun's Lair** (de la empresa ficticia Norsewind Studios) a un sistema de base de datos documental basado en **MongoDB**. La wiki permite a los jugadores consultar información sobre mazmorras, habitaciones, monstruos y tesoros del juego, además de publicar comentarios de cuatro tipos: *lore*, *hint*, *suggestion* y *bug*.

El prototipo trabaja con cuatro colecciones principales:

- **rooms**: Información de cada habitación, con sus monstruos, tesoros y comentarios embebidos.
- **users**: Datos de los usuarios y sus últimos comentarios.
- **monsters**: Catálogo de monstruos con referencia a las habitaciones donde aparecen.
- **loot**: Catálogo de objetos con referencia a las habitaciones donde se encuentran.

La práctica se divide en cuatro tareas: implementación de los endpoints REST, consultas analíticas con el pipeline de agregación, propuesta de patrones de diseño y, por último, una migración del esquema para centralizar los comentarios en una colección independiente.

2. Tarea 1: Implementación de los endpoints

Todas las funciones se conectan a MongoDB usando **PyMongo** y realizan el máximo de operaciones posibles dentro de la base de datos, evitando cálculos en Python cuando no es necesario.

Listing 1: Conexión inicial a MongoDB

```
from pymongo import MongoClient
from datetime import datetime

client = MongoClient("mongodb://localhost:27017/")
db = client["jotuns_lair"]

rooms = db["rooms"]
users = db["users"]
loot = db["loot"]
monsters = db["monsters"]
```

2.1. GET /user/{email}

Este endpoint devuelve todos los datos del usuario junto con sus 20 últimos comentarios, ordenados por fecha de creación descendente. Para ordenar el array de comentarios directamente en MongoDB usamos **\$sortArray** (disponible desde MongoDB 5.2), evitando así cualquier postprocesado en Python.

Listing 2: Endpoint GET /user/{email}

```
def get_user(email: str):
    """GET /user/{email}: devuelve el usuario con sus 20 ultimos comentarios."""
    pipeline = [
        {"$match": {"email": email}},
        {"$addFields": {
            "hints": {
                "$slice": [
```

```

        {"$sortArray": {
            "input": "$hints",
            "sortBy": {"creation_date": -1}
        }},
        20
    ]
}
}},
{"$project": {
    "_id": 0,
    "email": 1,
    "user_name": 1,
    "creation_date": 1,
    "country": 1,
    "hints": {
        "text": 1,
        "creation_date": 1,
        "category": 1,
        "referemces_room.room_id": 1,
        "referemces_room.room_name": 1,
        "referemces_room.dungeon_id": 1,
        "referemces_room.dungeon_name": 1
    }
}}
]
result = list(users.aggregate(pipeline))
return result[0] if result else None

```

Notamos que en los JSON proporcionados el campo se llama `referemces_rooms` (con errata), por lo que mantenemos ese nombre para ser consistentes con los datos reales.

2.2. GET /room/{room_id}

Devuelve la información básica de la habitación, el total de oro de sus tesoros, el recuento de monstruos por tipo (consultando la colección `monsters` mediante `$lookup`) y los 20 últimos comentarios. El cálculo del oro y el recuento de monstruos se realizan en su totalidad dentro del pipeline.

Listing 3: Endpoint GET /room/{room_id}

```

def get_room(room_id: int):
    """GET /room/{room_id}: habitacion con monstruos, oro y ultimos come
    ntarios."""
    pipeline = [
        {"$match": {"room_id": room_id}},
        {"$addFields": {
            "treasure_tot_value": {"$sum": "$loot.gold"},
            "hint": {
                "$slice": [
                    {"$sortArray": {
                        "input": "$hints",
                        "sortBy": {"creation_date": -1}
                    }},
                    20
                ]
            }
        }},
        {"$lookup": {
            "from": "monsters",

```

```

        "pipeline": [
            {"$unwind": "$in_rooms"},
            {"$match": {"in_rooms.room_id": room_id}},
            {"$group": {
                "_id": "$type",
                "count": {"$sum": "$in_rooms.amount"}
            }},
            {"$project": {"_id": 0, "type": "$_id", "count": 1}}
        ],
        "as": "monsters_info"
    }},
    {"$project": {
        "_id": 0,
        "room_id": 1,
        "room_name": 1,
        "in_waypoint": 1,
        "out_waypoint": 1,
        "monsters_info": 1,
        "treasure_tot_value": 1,
        "hint": {
            "creation_date": 1,
            "hintText": 1,
            "category": 1,
            "publish_by": {
                "user_name": 1,
                "country": 1,
                "creation_date": 1
            }
        }
    }
    }
}

]
result = list(rooms.aggregate(pipeline))
return result[0] if result else None

```

2.3. GET /dungeon/{dungeon_id}

Debe devolver información de la mazmorra junto con los datos necesarios para construir un grafo interactivo (nodos de habitaciones, aristas de conexiones, monstruos, tesoros y recuento de comentarios por categoría en cada sala).

Dado que los datos de mazmorra no están en una colección propia, partimos de `rooms` y hacemos un `$lookup` a sí misma para obtener la información del grafo. La nota del enunciado indicaba que el campo `lore` existe en la base de datos (pero no están en los datos proporcionados), por lo que no se incluye.

Listing 4: Endpoint GET /dungeon/{dungeon_id} (simplificado)

```

def get_dungeon(dungeon_id: int):
    """GET /dungeon/{dungeon_id}: info de mazmorra con grafo interactivo"""
    pipeline = [
        {"$match": {"dungeon_id": dungeon_id}},
        {"$group": {
            "_id": "$dungeon_id",
            "dungeon_name": {"$first": "$dungeon_name"},
            "rooms_list": {"$push": {
                "room_id": "$room_id",
                "room_name": "$room_name",
                "connections": "$rooms_connected",
            }}
        }}
    ]

```

```

        "monsters": {
            "$map": {
                "input": "$monsters",
                "as": "m",
                "in": {"id": "$$m.id", "name": "$$m.name"}
            }
        },
        "loot": {
            "$map": {
                "input": "$loot",
                "as": "l",
                "in": {"id": "$$l.id", "name": "$$l.name"}
            }
        },
        "hints_count": {
            "$size": {"$ifNull": ["$hints", []]}
        }
    }}
    {"$project": {
        "_id": 0,
        "dungeon_id": "$_id",
        "dungeon_name": 1,
        "rooms": "$rooms_list"
    }}
]
result = list(rooms.aggregate(pipeline))
return result[0] if result else None

```

2.4. POST /comment

Inserta un nuevo comentario tanto en el array `hints` de la habitación correspondiente como en el del usuario, manteniendo la consistencia del modelo embebido original.

Listing 5: Endpoint POST /comment

```

def post_comment(user_email: str, room_id: int,
                 text: str, category: str) -> dict:
    """POST /comment/: inserta comentario en rooms y en users."""
    user = users.find_one({"email": user_email})
    if not user:
        return {"ok": False, "error": "User not found"}

    comment_room = {
        "hintText": text,
        "creation_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "category": category,
        "publish_by": {
            "user_id": user.get("email"),
            "user_name": user.get("user_name"),
            "country": user.get("country"),
            "creation_date": user.get("creation_date")
        }
    }
    res_room = rooms.update_one(
        {"room_id": room_id},
        {"$push": {"hints": comment_room}}
    )

```

```

if res_room.matched_count == 0:
    return {"ok": False, "error": "Room not found"}

room = rooms.find_one({"room_id": room_id})
users.update_one(
    {"email": user_email},
    {"$push": {"hints": {
        "text": text,
        "creation_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "category": category,
        "referemces_room": {
            "room_id": room_id,
            "room_name": room.get("room_name"),
            "dungeon_id": room.get("dungeon_id"),
            "dungeon_name": room.get("dungeon_name")
        }
    }}}
)
return {"ok": True, "message": "Comment posted", "room_id": room_id}

```

2.5. DELETE /monster/{monster_id}

Elimina el monstruo de la colección `monsters` y además lo extrae del array embebido en todas las habitaciones donde aparecía, usando `$pull`.

Listing 6: Endpoint DELETE /monster/{monster_id}

```

def delete_monster(monster_id: int) -> dict:
    """DELETE /monster/{monster_id}: borra el monstruo de BD y de rooms.
    """
    res = monsters.delete_one({"id": monster_id})
    if res.deleted_count == 0:
        return {"ok": False, "error": "Monster not found"}

    res_rooms = rooms.update_many(
        {"monsters.id": monster_id},
        {"$pull": {"monsters": {"id": monster_id}}}
    )
    return {
        "ok": True,
        "message": (
            f"Monster {monster_id} deleted from "
            f"{res_rooms.modified_count} rooms"
        )
    }

```

3. Tarea 2: Consultas analíticas

Las siguientes consultas se han elaborado con MongoDB Compass usando `db.collection.aggregate()`. Mostramos el pipeline completo de cada una junto con un extracto de los resultados obtenidos.

3.1. Query 1: Monstruos en las 10 habitaciones con más comentarios Bug

Queremos obtener los monstruos (sin repeticiones) que aparecen en las 10 habitaciones con mayor número de comentarios de tipo *bug*. La estrategia es: calcular el número de bugs por

habitación con `$filter`, ordenar y limitar a 10, desanidar el array de monstruos con `$unwind` y agrupar para eliminar duplicados.

Colección: rooms

Listing 7: Pipeline Query 1

```
[
  { $match: { hints: { $ne: null } } },
  { $addFields: {
    n_hints_bugs: {
      $size: {
        $filter: {
          input: "$hints",
          as: "hint",
          cond: { $eq: ["$$hint.category", "bug"] }
        }
      }
    }
  } },
  { $sort: { n_hints_bugs: -1 } },
  { $limit: 10 },
  { $unwind: "$monsters" },
  { $group: {
    _id: "$monsters.id",
    name: { $first: "$monsters.name" }
  } }
]
```

Resultado (extracto):

```
{ "_id": 265, "name": "sea hag" }
{ "_id": 249, "name": "kuo-toa archpriest" }
{ "_id": 116, "name": "vine blight" }
{ "_id": 79, "name": "couatl" }
{ "_id": 117, "name": "driad" }
{ "_id": 258, "name": "quaggoth" }
{ "_id": 264, "name": "chuul" }
{ "_id": 247, "name": "hook horror" }
```

La consulta devuelve 8 monstruos distintos. Que sean tan pocos para 10 habitaciones indica que varias de esas salas comparten criaturas, o que ciertos enemigos como *sea hag* o *kuo-toa archpriest* aparecen simultáneamente en múltiples salas problemáticas. Desde la perspectiva del equipo de QA, estos monstruos son los candidatos prioritarios a revisar en busca de comportamientos erróneos que estén generando los reportes de bug.

3.2. Query 2: Mazmorras con comentarios Hint por encima de la media

El reto aquí es comparar cada mazmorra con la media global en una sola consulta. La solución es calcular la media global en una etapa `$group` con `_id: null` y luego usar `$unwind` para volver a tener una fila por mazmorra y filtrar con `$expr`.

Colección: rooms

Listing 8: Pipeline Query 2

```
[
  { $addFields: {
    n_hints: {
      $size: {
        $filter: {
          input: "$hints",
```

```

        as: "hint",
        cond: { $eq: ["$$hint.category", "hint"] }
      }
    }
  }
}},
{ $group: {
  _id: "$dungeon_id",
  dungeon_name: { $first: "$dungeon_name" },
  total_hints_mazmorra: { $sum: "$n_hints" }
}},
{ $group: {
  _id: null,
  media_global: { $avg: "$total_hints_mazmorra" },
  mazmorras: {
    $push: {
      nombre: "$dungeon_name",
      total: "$total_hints_mazmorra"
    }
  }
}},
{ $unwind: "$mazmorras" },
{ $match: { $expr: { $gt: ["$mazmorras.total", "$media_global"] } } },
{ $project: { _id: 0, dungeon_name: "$mazmorras.nombre" } }
]

```

Resultado (extracto):

```

{ "dungeon_name": "Alaunstream, Pit of the Jolly Scientists"      }
{ "dungeon_name": "Wanton, Culverts of the Jealous Thieves"     }
{ "dungeon_name": "Dalshady, Cave of the Old-Fashioned Otaku"   }
{ "dungeon_name": "Burghap, Prison of the Jealous Hippies"     }
{ "dungeon_name": "Fenwater, Hobble of the Fierce Furries"     }
{ "dungeon_name": "Arvaleclock, Dungeon of the Snobbish Scientists" }
{ "dungeon_name": "Corrieclock, Pit of the Sexy Unknowns"      }
{ "dungeon_name": "Marshgreat, Catacombs of the Wandering Degenerates" }
{ "dungeon_name": "Greatcliffe, Castle of the Magnificent Sumo Wrestlers" }
{ "dungeon_name": "Watergarth, Stockade of the Feminist Presidents" }

```

Se obtienen 10 mazmorras cuyo volumen de comentarios de tipo *hint* supera la media del juego. Son las mazmorras donde los jugadores sienten más necesidad de compartir trucos y consejos, lo que probablemente indica que son las más difíciles o las que tienen mecánicas menos intuitivas. Esta información es útil para el equipo de diseño de niveles: si una mazmorra acumula muchos hints puede ser que necesite algún ajuste de dificultad o de comunicación al jugador.

3.3. Query 3: Estadísticas por mazmorra

Esta es la consulta más extensa. Para cada mazmorra hay que devolver en una sola query: comentarios por tipo, oro total, nivel mediano de monstruos y usuarios por país. Lo resolvemos con cuatro `$lookup` anidados sobre la propia colección `rooms`, cada uno especializado en calcular uno de los campos pedidos. Para el nivel mediano usamos el operador `$percentile` con `p: [0.5]`.

Colección: `rooms`

Listing 9: Pipeline Query 3 (estructura general)

```

[
  // Paso 1: obtener mazmorras unicas
  { $group: {

```

```

    _id: "$dungeon_id",
    dungeon_name: { $first: "$dungeon_name" }
  }},

// Paso 2: comentarios por tipo via $lookup
{ $lookup: {
  from: "rooms",
  let: { d_id: "$_id" },
  pipeline: [
    { $match: { $expr: { $eq: ["$dungeon_id", "$$d_id"] } } },
    { $unwind: { path: "$hints",
      preserveNullAndEmptyArrays: false } },
    { $group: { _id: "$hints.category", total: { $sum: 1 } } },
    { $project: { _id: 0, tipo: "$_id", total: 1 } }
  ],
  as: "comentarios_por_tipo"
}},

// Paso 3: usuarios por pais via $lookup
{ $lookup: {
  from: "rooms",
  let: { d_id: "$_id" },
  pipeline: [
    { $match: { $expr: { $eq: ["$dungeon_id", "$$d_id"] } } },
    { $unwind: "$hints" },
    { $group: {
      _id: { email: "$hints.publish_by.email",
        pais: "$hints.publish_by.country" }
    }},
    { $group: { _id: "$_id.pais", total: { $sum: 1 } } },
    { $project: { _id: 0, pais: "$_id", total: 1 } }
  ],
  as: "usuarios_por_pais"
}},

// Paso 4: nivel mediano ($percentile p=0.5)
{ $lookup: {
  from: "rooms",
  let: { d_id: "$_id" },
  pipeline: [
    { $match: { $expr: { $eq: ["$dungeon_id", "$$d_id"] } } },
    { $unwind: "$monsters" },
    { $group: {
      _id: null,
      nivel_mediano_array: {
        $percentile: {
          p: [0.5],
          input: "$monsters.level",
          method: "approximate"
        }
      }
    }},
    { $project: {
      _id: 0,
      nivel_mediano: { $arrayElemAt: ["$nivel_mediano_array", 0] }
    }
  ],
  as: "datos_nivel"
}

```

```

}},

// Paso 5: oro total
{ $lookup: {
  from: "rooms",
  let: { d_id: "$_id" },
  pipeline: [
    { $match: { $expr: { $eq: ["$dungeon_id", "$$d_id"] } } },
    { $unwind: "$loot" },
    { $group: { _id: null, oro_total: { $sum: "$loot.gold" } } },
    { $project: { _id: 0, oro_total: 1 } }
  ],
  as: "datos_oro"
}},

// Paso 6: proyeccion final
{ $project: {
  _id: 0,
  dungeon_name: 1,
  comentarios_por_tipo: 1,
  usuarios_por_pais: 1,
  nivel_mediano: {
    $ifNull: [{ $arrayElemAt: ["$datos_nivel.nivel_mediano", 0] },
    0]
  },
  oro_total: {
    $ifNull: [{ $arrayElemAt: ["$datos_oro.oro_total", 0] }, 0]
  }
}}
]

```

Resultado (extracto de una mazmorra):

```

{
  "dungeon_name": "Watergarth, Stockade of the Feminist Presidents",
  "comentarios_por_tipo": [
    { "tipo": "suggestion", "total": 19 },
    { "tipo": "hint", "total": 196 },
    { "tipo": "lore", "total": 404 },
    { "tipo": "bug", "total": 208 }
  ],
  "nivel_mediano": 6.0,
  "oro_total": 8450,
  "usuarios_por_pais": [
    { "pais": "ES", "total": 34 },
    { "pais": "US", "total": 28 }
  ]
}

```

El resultado muestra un perfil completo de cada mazmorra en una sola consulta. En el ejemplo de *Watergarth, Stockade of the Feminist Presidents*, el comentario de tipo *lore* es con diferencia el más frecuente (404), lo que sugiere que esta mazmorra tiene un trasfondo narrativo muy rico que despierta el interés de los jugadores. El campo *bug* con 208 entradas es notable y debería llamar la atención del equipo de QA. El nivel mediano de 6 indica que los monstruos son de dificultad moderada, mientras que el oro total de 8450 da una idea del valor económico de explorar esa mazmorra al completo.

3.4. Query 4: Mazmorras por tipo de monstruo

Para cada tipo de monstruo devolvemos un array con todas las mazmorras en las que aparece (sin repeticiones). Usamos `$addToSet` en el `$group` para garantizar la unicidad.

Colección: `monsters`

Listing 10: Pipeline Query 4

```
[
  { $unwind: "$in_rooms" },
  { $group: {
    _id: "$type",
    dungeons: { $addToSet: "$in_rooms.dungeon_name" }
  }},
  { $project: {
    _id: 0,
    monster_type: "$_id",
    dungeons: 1
  }}
]
```

Resultado (extracto):

```
{
  "monster_type": "fey",
  "dungeons": [
    "Dalshady, Cave of the Old-Fashioned Otaku",
    "Alaunstream, Pit of the Jolly Scientists",
    "Withystream, Castle of the Screeching Otaku",
    "Wanton, Culverts of the Jealous Thieves",
    ...
  ]
}
```

El resultado permite construir un mapa de distribución de los tipos de monstruo a lo largo del juego. Observamos que el tipo *fey* aparece en un número elevado de mazmorras, lo que sugiere que es uno de los tipos más extendidos en el mundo del juego. Esta consulta es especialmente útil para el equipo de contenidos: si se elimina un tipo de monstruo o se introduce uno nuevo, se puede saber de un vistazo cuantas mazmorras se verán afectadas.

3.5. Query 5: Tipo de monstruo más común por mazmorra

Agrupamos por (mazmorra, tipo) sumando las cantidades con `$sum: "$in_rooms.amount"`, ordenamos descendente por cantidad y usamos `$first` en el segundo `$group` para quedarnos con el tipo más abundante. En caso de empate se devuelve uno de los tipos empatados.

Colección: `monsters`

Listing 11: Pipeline Query 5

```
[
  { $unwind: "$in_rooms" },
  { $group: {
    _id: { dungeon: "$in_rooms.dungeon_name", type: "$type" },
    cantidad_total: { $sum: "$in_rooms.amount" }
  }},
  { $sort: { "_id.dungeon": 1, cantidad_total: -1 } },
  { $group: {
    _id: "$_id.dungeon",
    tipo_mas_comun: { $first: "$_id.type" }
  }}
]
```

```

{ $project: {
  _id: 0,
  dungeon_name: "$_id",
  tipo_mas_comun: 1
}}
]

```

Resultado (extracto):

```

{ "tipo_mas_comun": "monstrosity",
  "dungeon_name": "Dalhylls, Culverts of the Grumpy Bandits" }
{ "tipo_mas_comun": "fey",
  "dungeon_name": "Bentalaun, Vault of the Old-Fashioned Pirates" }
{ "tipo_mas_comun": "aberration",
  "dungeon_name": "Burghap, Prison of the Jealous Hippies" }
{ "tipo_mas_comun": "undead",
  "dungeon_name": "Schyppegarth, Laboratory of the Sticky Robots" }

```

El resultado cubre las 20 mazmorras del juego. El tipo *monstrosity* es el mas comun: aparece en 6 de las 20 mazmorras, lo que lo convierte en la categoria dominante del bestiario. Le siguen *fey*, *humanoid* y *undead* con presencia en varias mazmorras. Curiosamente, *aberration* y *goblinoid* solo dominan en una mazmorra cada uno, lo que los hace raros y posiblemente ligados a mazmorras temáticas concretas. Este tipo de distribución es útil para garantizar variedad en el diseño de contenido.

3.6. Query 6: Comentarios de la habitación con encuentro de mayor nivel medio

Calculamos el nivel medio de cada habitación con \$avg: "\$monsters.level", obtenemos el máximo global con \$max, y filtramos las habitaciones que igualen ese máximo. Si varias habitaciones empatan, se devuelven todas.

Colección: rooms

Listing 12: Pipeline Query 6

```

[
{ $match: { monsters: { $ne: null } } },
{ $addFields: {
  nivel_medio_encuentro: { $avg: "$monsters.level" }
}},
{ $group: {
  _id: null,
  maximo_absoluto: { $max: "$nivel_medio_encuentro" },
  habitaciones: {
    $push: {
      nivel_medio: "$nivel_medio_encuentro",
      comentarios: "$hints",
      room_id: "$room_id"
    }
  }
}},
{ $unwind: "$habitaciones" },
{ $match: {
  $expr: {
    $eq: ["$habitaciones.nivel_medio", "$maximo_absoluto"]
  }
}},
{ $project: {
  _id: 0,

```

```

    comentarios: "$habitaciones.comentarios",
    room_id:     "$habitaciones.room_id"
  }}
]

```

El resultado devuelve el `room_id` de la habitación con el encuentro más difícil junto con todos sus comentarios. El JSON resultante (q6.json, 39 KB) contiene una cantidad muy elevada de comentarios, lo que indica que esta habitación ha generado gran actividad en la wiki: los jugadores que la alcanzan tienden a dejar mucho feedback, probablemente porque el encuentro supone un salto de dificultad significativo respecto al resto del juego. La variedad de categorías de comentarios (hints con estrategias, bugs, lore) refleja que es una habitación muy relevante tanto jugablemente como narrativamente. Por el volumen del JSON no reproducimos el extracto completo aquí.

3.7. Query 7: Habitaciones outliers según ratio oro/nivel

La ratio oro-nivel se define como $\text{ratio} = \frac{\text{oro total}}{\text{nivel medio}}$. Una habitación es *outlier leve* si su ratio cumple:

$$\text{ratio} < Q_1 - 1.5 \cdot \text{IQR} \quad \text{o} \quad \text{ratio} > Q_3 + 1.5 \cdot \text{IQR}$$

donde $\text{IQR} = Q_3 - Q_1$.

Usamos el operador `$percentile` con `p`: [0.25, 0.75] para calcular los cuartiles sobre el array acumulado de ratios. Solo consideramos habitaciones que tengan tanto tesoros como monstruos (para evitar divisiones por cero).

Colección: rooms

Listing 13: Pipeline Query 7

```

[
  { $match: { loot: { $ne: null }, monsters: { $ne: null } } },
  { $addFields: {
    total_oro: { $sum: "$loot.gold" },
    nivel_medio: { $avg: "$monsters.level" }
  }},
  { $addFields: {
    ratio_oro_nivel: { $divide: ["$total_oro", "$nivel_medio"] }
  }},
  { $sort: { ratio_oro_nivel: 1 } },
  { $group: {
    _id: null,
    ratio_array: { $push: "$ratio_oro_nivel" },
    rooms: {
      $push: { _id: "$_id", ratio: "$ratio_oro_nivel",
        room_id: "$room_id" }
    }
  }},
  { $addFields: {
    cuartiles: {
      $percentile: {
        input: "$ratio_array",
        p: [0.25, 0.75],
        method: "approximate"
      }
    }
  }},
  { $addFields: {
    q1: { $arrayElemAt: ["$cuartiles", 0] },
    q3: { $arrayElemAt: ["$cuartiles", 1] }
  }
}

```

```

    }},
    { $addField: {
      iqr: { $subtract: ["$q3", "$q1"] }
    }},
    { $addField: {
      lower: { $subtract: ["$q1", { $multiply: [1.5, "$iqr"] }] },
      upper: { $add: ["$q3", { $multiply: [1.5, "$iqr"] }] }
    }},
    { $unwind: "$rooms" },
    { $match: {
      $expr: {
        $or: [
          { $lt: ["$rooms.ratio", "$lower"] },
          { $gt: ["$rooms.ratio", "$upper"] }
        ]
      }
    }},
    { $project: {
      _id: 0,
      room_id: "$rooms.room_id",
      ratio: "$rooms.ratio"
    }
  }
}
]

```

Resultado:

```

{ "room_id": 208, "ratio": 8162.53 }
{ "room_id": 604, "ratio": 8162.53 }
{ "room_id": 183, "ratio": 8616.0 }
{ "room_id": 307, "ratio": 8616.0 }
{ "room_id": 31, "ratio": 8616.0 }
{ "room_id": 96, "ratio": 8616.0 }
{ "room_id": 631, "ratio": 8616.0 }
{ "room_id": 556, "ratio": 9693.0 }
{ "room_id": 394, "ratio": 11488.0 }
{ "room_id": 547, "ratio": 13283.0 }
{ "room_id": 568, "ratio": 13283.0 }
{ "room_id": 718, "ratio": 13283.0 }

```

Todos los outliers identificados tienen ratios **por encima** del límite superior, es decir, no hay ninguna habitación con ratio anormalmente bajo. Esto significa que no existen salas donde los monstruos sean muy poderosos y el botín escaso, pero sí hay 12 habitaciones donde el oro acumulado es desproporcionadamente alto en relación a la dificultad del encuentro. Casos como las habitaciones 183, 307, 31, 96 y 631 (todas con ratio 8616) podrían representar salas idénticas o generadas con el mismo patrón. Los valores más extremos (13283, en las habitaciones 547, 568 y 718) son claramente anómalos y podrían indicar un error en la generación del loot o una decisión de diseño intencionada para ofrecer recompensas especiales.

4. Tarea 3: Patrones de diseño

Teniendo en cuenta los datos de uso de los endpoints:

Tipo	Endpoint	Frecuencia
Lectura	GET /dungeon/{dungeon_id}	1 000 000/día
Lectura	GET /room/{room_id}	500 000/día
Lectura	GET /user/{email}	300 000/día
Escritura	POST /comment	10 000/día
Escritura	DELETE /monster/{monster_id}	100/año

Proponemos tres patrones complementarios:

4.1. Patrón Subset (Subconjunto)

Problema: Los documentos de **rooms** y **users** contienen arrays **hints** que crecen ilimitadamente, aumentando el tamaño del documento y penalizando las lecturas frecuentes.

Aplicación:

- **Colección Rooms:** Actualmente el array **hints** almacena **todos** los comentarios de una sala. El endpoint GET /room/{room_id} solo necesita los últimos 20 comentarios. Por tanto, es mejor eliminar el array completo de la sala y, si se desea máxima velocidad, mantener solo un subconjunto con los 20 comentarios más recientes. El historial completo se mueve a una colección independiente **Hints**, con el histórico de comentarios de cada sala. Con esto la sala ocupa mucho menos espacio y las lecturas frecuentes (500K/día) se sirven desde un solo documento más ligero.

Independiente de que tengamos que escribir en **Hints** cada vez que se añade un comentario, el volumen de escritura (10K/día) es manejable y no afecta significativamente el rendimiento.

- **Colección Users:** El endpoint GET /user/{email} (300K/día) devuelve los últimos 20 comentarios del usuario. Si los comentarios solo residieran en **Hints**, habría que hacer una consulta con \$lookup o un find sobre **Hints** ordenado y limitado, lo que añade latencia. Por ello, es mejor mantener un array **last_20_comments** en el documento del usuario, que se actualiza cada vez que se publica un nuevo comentario (usando \$push con \$slice para conservar solo los 20 más recientes). Así la lectura de usuario es directa.
- También se puede crear una nueva colección **Dungeon** para poder almacenar la información que requiere el endpoint GET /dungeon/{dungeon_id} (1M/día, la más consultada) de forma optimizada, sin necesidad de realizar operaciones de agregación complejas (teníamos que utilizar la colección de **Rooms** para poder obtener la información necesaria). Esta tendría que estar organizada por **dungeon_id**, con un documento por cada mazmorra, y con la información de las habitaciones (comentarios, monstruos, loot...) anidada dentro del documento de la mazmorra. De esta forma, el endpoint GET /dungeon/{dungeon_id} se puede servir con una sola consulta a la colección **Dungeon**. Además, apenas hay que actualizar esta colección, ya que solo se actualiza cuando se añaden nuevos comentarios o monstruos a las habitaciones, lo cual es un volumen de escritura perfectamente asumible (10K/día).

El subconjunto se sincroniza en las escrituras, cuyo volumen (10K/día y 100 año) es perfectamente asumible.

El volumen de escritura (10K/día) es lo suficientemente bajo como para que la sincronización de los subconjuntos en cada POST /comment sea perfectamente asumible.

4.2. Patron Computed (Valores precalculados)

Problema: Calcular el numero de comentarios por categoria, el total de oro o el recuento de monstruos por tipo en tiempo de lectura supone recorrer arrays anidados en cada una de las 1M + 500K peticiones diarias, lo que es muy costoso.

Aplicación:

- **Endpoint** GET /room/{room_id} (500K/día): Devuelve el número de monstruos de cada tipo y el total de oro de los tesoros de la sala. Podríamos añadir campos precalculados como `total_gold` y `monster_count_by_type` en el documento de la sala. Se actualizan únicamente cuando se modifica la composición de monstruos/tesoros de la sala (operación rarísima), evitando recorrer arrays anidados en cada lectura.
- **Endpoint** GET /dungeon/{dungeon_id} (1M/día): Necesita el número de comentarios de cada categoría para cada sala de la mazmorra. Parecido al anterior, podemos almacenar en el documento de cada sala (o en el de la mazmorra) un objeto `comments_by_category` (p. ej. {bug: 34, hint: 12, suggestion: 5, lore: 8}). Este contador se incrementa atómicamente con cada POST /comment (solo 10K/día), de modo que la consulta de mazmorra no tenga que agregar sobre la colección `Hints` en tiempo real.
- Es mejor calcular una vez al escribir que repetir el cálculo millones de veces al leer. Dado que el ratio lectura/escritura es aproximadamente 100:1, el coste de actualizar estos contadores en escritura es mínimo frente al beneficio de eliminar las agregaciones en lectura.

4.3. Patron Outlier (Datos atípicos)

Problema: Algunos documentos de `rooms` tienen arrays `loot` o `monsters` excepcionalmente grandes, mientras que otras habitaciones no tienen ninguno. Esta variabilidad hace que el tamaño de los documentos sea muy heterogéneo.

Aplicación:

- Mover `loot` y `monsters` a colecciones independientes cuando superen un umbral definido, añadiendo un flag `is_outlier: true` al documento de la sala.
- Las salas normales mantienen sus arrays embebidos (lectura rápida). Las salas outlier realizan consultas adicionales a las colecciones externas sólo cuando es necesario.
- Dado que los comentarios ya se han limitado a 20 mediante el patrón Subset, no representan un riesgo de crecimiento descontrolado.

4.4. Justificación global

La combinación de estos tres patrones permite que:

1. Las lecturas de `dungeon`, `room` y `user` se sirvan desde uno o dos documentos como máximo, sin agregaciones costosas en tiempo real.
2. El único punto de escritura significativa (POST /comment) actualiza de forma controlada los subconjuntos y contadores precalculados con operaciones atómicas.
3. Los documentos atípicos se gestionan de forma diferenciada para no penalizar al resto.

5. Tarea 4: Cambio de esquema

Esta tarea consiste en centralizar todos los comentarios en una nueva colección `Hints`, eliminar el campo `hints` de `rooms` y `users`, y adaptar los endpoints.

5.1. A. Consulta de extraccion y exportacion a JSON

Extraemos los comentarios directamente de `rooms`, ya que cada comentario contiene la información del usuario en su subdocumento `publish_by`. Esto evita posibles duplicados o inconsistencias entre los arrays de `rooms` y `users`.

Listing 14: Pipeline de extraccion de comentarios

```
db.rooms.aggregate([
  { $unwind: "$hints" },
  { $project: {
    _id: 0,
    Creation_date: "$hints.creation_date",
    HintText: "$hints.hintText",
    Category: "$hints.category",
    References_room: {
      IdR: "$room_id",
      Name: "$room_name",
      IdD: "$dungeon_id",
      Dungeon: "$dungeon_name"
    },
    Publish_by: {
      Email: "$hints.publish_by.email",
      User_name: "$hints.publish_by.user_name",
      CreationDate: "$hints.publish_by.creation_date",
      Country: "$hints.publish_by.country"
    }
  }}
])
```

El resultado se exporta a `hints.json` desde MongoDB Compass (boton *Export Data*) o mediante un script Python que escriba el cursor en un fichero.

5.2. B. Crear la coleccion Hints y limpiar las originales

Listing 15: Importar hints.json con mongoimport

```
mongoimport --db jotuns_lair --collection Hints \
  --file hints.json --jsonArray
```

Una vez verificada la importación, eliminamos el campo `hints` de las colecciones originales:

Listing 16: Eliminar el campo hints de rooms y users

```
db.rooms.updateMany({}, { $unset: { hints: "" } });
db.users.updateMany({}, { $unset: { hints: "" } });
```

En Python:

Listing 17: Eliminar hints en Python

```
def eliminar_hints():
    res_rooms = rooms.update_many({}, {"$unset": {"hints": ""}})
    res_users = users.update_many({}, {"$unset": {"hints": ""}})
    return res_rooms.modified_count, res_users.modified_count
```

5.3. C. Adaptación de los endpoints

5.3.1. POST /comment (nuevo)

En lugar de hacer push en dos colecciones, ahora se inserta un único documento en `Hints`. Esto elimina la duplicación y hace la escritura atómica.

Listing 18: POST /comment con coleccion Hints

```

hints_col = db["Hints"]

def post_comment(user_email, room_id, text, category):
    user = users.find_one({"email": user_email})
    room = rooms.find_one({"room_id": room_id})
    new_comment = {
        "Creation_date": datetime.utcnow().isoformat(),
        "HintText": text,
        "Category": category,
        "References_room": {
            "IdR": room["room_id"],
            "Name": room["room_name"],
            "IdD": room["dungeon_id"],
            "Dungeon": room["dungeon_name"]
        },
        "Publish_by": {
            "Email": user_email,
            "User_name": user["user_name"],
            "CreationDate": user["creation_date"],
            "Country": user["country"]
        }
    }
    hints_col.insert_one(new_comment)

```

5.3.2. GET /room/{room_id} (nuevo)

Los comentarios ya no estan embebidos. Se añade un \$lookup sobre Hints filtrando por References_room.IdR, ordenando por fecha y limitando a 20.

Listing 19: GET /room con lookup a Hints

```

def get_room(room_id: int):
    pipeline = [
        {"$match": {"room_id": room_id}},
        {"$lookup": {
            "from": "Hints",
            "pipeline": [
                {"$match": {"References_room.IdR": room_id}},
                {"$sort": {"Creation_date": -1}},
                {"$limit": 20},
                {"$project": {
                    "_id": 0,
                    "Creation_date": 1,
                    "HintText": 1,
                    "Category": 1,
                    "Publish_by": {
                        "User_name": 1, "Country": 1,
                        "CreationDate": 1
                    }
                }}
            ]
        }},
        {"as": "hint"}
    ],
    {"$lookup": {
        "from": "monsters",
        "pipeline": [
            {"$unwind": "$in_rooms"},

```

```

        {"$match": {"in_rooms.room_id": room_id}},
        {"$group": {"_id": "$type",
                     "count": {"$sum": "$in_rooms.amount"}}},
        {"$project": {"_id": 0, "type": "$_id", "count": 1}}
    ],
    "as": "monsters_info"
}},
{"$project": {
    "_id": 0, "room_id": 1, "room_name": 1,
    "in_waypoint": 1, "out_waypoint": 1,
    "treasure_tot_value": {"$sum": "$loot.gold"},
    "monsters_info": 1, "hint": 1
}}
]
result = list(rooms.aggregate(pipeline))
return result[0] if result else None

```

5.3.3. GET /user/{email} (nuevo)

Similar al caso anterior: se sustituye el array embebido por un \$lookup sobre Hints filtrando por Publish_by.Email.

Listing 20: GET /user con lookup a Hints

```

def get_user(email: str):
    pipeline = [
        {"$match": {"email": email}},
        {"$lookup": {
            "from": "Hints",
            "pipeline": [
                {"$match": {"Publish_by.Email": email}},
                {"$sort": {"Creation_date": -1}},
                {"$limit": 20},
                {"$project": {
                    "_id": 0, "HintText": 1,
                    "Creation_date": 1, "Category": 1,
                    "References_room.IdR": 1,
                    "References_room.Name": 1,
                    "References_room.IdD": 1,
                    "References_room.Dungeon": 1
                }}
            ],
            "as": "hint"
        }},
        {"$project": {
            "_id": 0, "email": 1, "user_name": 1,
            "creation_date": 1, "country": 1, "hint": 1
        }}
    ]
    result = list(users.aggregate(pipeline))
    return result[0] if result else None

```

5.3.4. GET /dungeon/{dungeon_id} (nuevo)

El endpoint de mazmorra ahora consulta Hints para obtener el recuento de comentarios por categoría en cada habitación. Si se hubiera aplicado el patrón Computed, este cálculo ya estaría precalculado y no haría falta la consulta adicional.

Listing 21: GET /dungeon con lookup a Hints

```
def get_dungeon(dungeon_id):
    pipeline = [
        {
            "$match": { "dungeon_id": dungeon_id }
        },
        {
            "$lookup": {
                "from": "Hints",
                "localField": "room_id",
                "foreignField": "References_room.IdR",
                "as": "room_hints"
            }
        },
        {
            "$addFields": {
                "monsters_graph": {
                    "$setUnion": [
                        {
                            "$map": {
                                "input": { "$ifNull": ["$monsters", []] },
                                "as": "m",
                                "in": { "id": "$$m.id", "name": "$$m.name" }
                            }
                        }
                    ]
                },
                "loot_graph": {
                    "$setUnion": [
                        {
                            "$map": {
                                "input": { "$ifNull": ["$loot", []] },
                                "as": "l",
                                "in": { "id": "$$l.id", "name": "$$l.name" }
                            }
                        }
                    ]
                },
                "comments_summary": {
                    "$map": {
                        "input": {
                            "$setUnion": [
                                {
                                    "$map": {
                                        "input": { "$ifNull": ["$room_hints", []] },
                                        "as": "h",
                                        "in": "$$h.Category"
                                    }
                                }
                            ]
                        },
                        "as": "hints_category",
                        "in": {
                            "category": "$$hints_category",

```

```

        "cantidad": {
            "$size": {
                "$filter": {
                    "input": { "$ifNull": ["$room_hints", []] },
                    "as": "h",
                    "cond": { "$eq": ["$$h.Category", "$$hints_category"] }
                }
            }
        }
    }
}

},
{
    "$group": {
        "_id": "$dungeon_id",
        "name": { "$first": "$dungeon_name" },
        "graph_rooms": {
            "$push": {
                "room_id": "$room_id",
                "room_name": "$room_name",
                "connections": { "$ifNull": ["$rooms_connected", []] },
                "monsters": "$monsters_graph",
                "loot": "$loot_graph",
                "comments_count": "$comments_summary"
            }
        }
    }
},
{
    "$project": {
        "_id": 0,
        "idD": "$_id",
        "name": 1,
        "rooms": "$graph_rooms"
    }
}
]

result = list(db.rooms.aggregate(pipeline))

return result[0] if result else {}

```

5.3.5. Impacto general del cambio

- **Ventajas:** los comentarios quedan centralizados y normalizados, facilitando las consultas analíticas sin recorrer arrays enormes. Las escrituras son atómicas sobre un único documento. Se elimina la duplicación entre **rooms** y **users**.
- **Desventajas y contramedidas:** las lecturas de comentarios requieren consultas adicionales a **Hints**. Se mitigan con índices compuestos adecuados:
 - {References_room.IdR: 1, Creation_date: -1} para GET /room.

- `{Publish_by.Email: 1, Creation_date: -1}` para `GET /user`.
 - `{References_room.IdD: 1, Category: 1}` para consultas analíticas por mazmorra.
- El endpoint mas crítico (`GET /dungeon`, 1M/día) no sufre penalización si se combinan con el patrón `Computed`, que mantiene los contadores precalculados en cada sala.

6. Conclusiones

A lo largo de la práctica hemos trabajado con un dataset real de un videojuego MMO para explorar las capacidades de MongoDB en un escenario típico de migración desde una base de datos relacional.

Las principales lecciones que extraemos son:

1. El **pipeline de agregación** es sumamente expresivo y permite resolver consultas muy complejas (como los outliers de la Query 7 o las estadísticas multidimensionales de la Query 3) sin salir de la base de datos, lo que es fundamental para el rendimiento.
2. El operador **\$percentile** (introducido en MongoDB 7.0) simplifica enormemente cálculos estadísticos que antes requerían soluciones manuales con `$sort` y `$arrayElemAt`.
3. Los **patrones de diseño** (`Subset`, `Computed`, `Outlier`) no son opcionales en sistemas con alta carga de lectura: sin ellos, el endpoint `GET /dungeon` con 1M de peticiones diarias sería inviable con agregaciones en tiempo real.
4. El **cambio de esquema** de la Tarea 4 (centralización de comentarios en `Hints`) refleja un compromiso clásico en MongoDB: ganar en normalización y simplicidad de escritura a costa de añadir consultas adicionales en lectura, compensadas con índices adecuados.
5. La elección entre **modelo embebido y referenciado** debe hacerse siempre en función de los patrones de acceso de la aplicación, no como una decisión puramente teórica.